GraphManager: Cost-Aware Graph-Guided Retrieval for Issue Localization in Software Repositories

Tristan Yan
Colorado School of Mines

March 12, 2026

Abstract

LLM-based automated program repair requires localizing relevant code within large repositories, yet the dominant retrieval-augmented generation (RAG) paradigm—embedding, indexing, and querying repository contents—incurs substantial cost that is rarely reported alongside resolve rates. We propose *Graph-Map Progressive* (GM-P), a lightweight context-construction strategy that builds a language-level dependency graph from the repository and progressively expands context from the suspected fault locale along structural edges. By replacing embedding-based retrieval with deterministic graph traversal, GM-P eliminates per-query vector-search costs while preserving the LLM's ability to reason over cross-file dependencies. On SWE-bench Verified (500 instances across 12 repositories), GM-P resolves 46.8% of issues—significantly more than every compared method after Holm–Bonferroni correction of paired McNemar tests ($\alpha = 0.05$)—against an oracle upper bound of 53.8%. Crucially, GM-P consumes approximately 256K tokens per resolved issue, roughly 7.5× fewer than the nearest RAG-based progressive method (1.93M tokens), because the graph is built once with deterministic tooling and amortized across all issues within a repository snapshot.[1] These results suggest that structural dependency graphs offer a cost-effective and statistically reliable alternative to embedding-based retrieval for repository-level issue localization and code repair.

1 Introduction

LLM-based automated program repair has advanced rapidly, with benchmarks such as SWE-bench [12] and its human-validated *Verified* subset becoming the standard yardstick for repository-level bug fixing. In this task an LLM receives a natural-language issue report and must produce a patch that passes the project's test suite—a setting that demands not only code-generation ability but also accurate *localization* of the relevant source files and functions within repositories that may contain thousands of files. Context construction—deciding which code to show the LLM—is therefore a critical bottleneck, and the strategies adopted by leading systems such as SWE-Agent [27], AutoCodeRover [28], and Agentless [26] differ primarily in *how* they select context rather than in the underlying language model.

The prevailing approach to context construction is retrieval-augmented generation (RAG) [14]: chunk the repository, embed every chunk, build a vector index, and retrieve the top-k fragments most similar to the issue text. While effective, this pipeline carries costs that are seldom reported. Embedding an entire repository requires per-token API calls or dedicated GPU time; the resulting index must be stored and, in continuous-integration settings, rebuilt whenever the codebase changes. Most SWE-bench studies report *resolve rate* as the sole metric, making it impossible to judge whether a method is practically deployable at scale. We argue that **context-construction cost deserves first-class status alongside resolve rate** when evaluating repository-level repair.

Programming languages, however, offer an alternative source of relevance signal that is both cheaper and more precise than semantic similarity: *structural dependency*. Import statements, function calls, class inheritance, and module boundaries define an explicit graph that can be extracted with deterministic, non-LLM tooling such as Tree-sitter [1] at negligible marginal cost. A dependency graph built once per repository snapshot can be traversed for every incoming issue without further embedding or indexing. Prior work on repository maps—notably Aider's `RepoMap` [6]—has explored structural summaries for LLM context, but no controlled study has systematically compared graph-guided retrieval against RAG under identical LLM, patching, and evaluation conditions.

In this paper we introduce *GraphManager* and its flagship strategy, **Graph-Map Progressive (GM-P)**. GM-P constructs a language-level dependency graph of the target repository, seeds the traversal from the suspected fault locale, and progressively expands context along structural edges—using agentic tool calls (`search_nodes`,

[1] Both graph and RAG indices were constructed in our experimental harness; reported costs account for method-specific token consumption only. See Section 7 for discussion.

`get_neighbors`, `get_file_summary`)—until a token budget is filled. We evaluate GM-P against 10 alternative context-construction strategies spanning BM25 lexical search, fixed- and progressive-window RAG, agentic cold-start exploration, and controlled re-implementations inspired by Agentless-style localization [26] and Aider-style repository maps [6]. All 11 methods share the same downstream LLM (Gemini [8]), the same patching pipeline, and the same evaluation harness across 500 SWE-bench Verified instances drawn from 12 open-source repositories. We report both resolve rate and per-resolved-issue token cost, and we assess pairwise differences with McNemar's test [19] corrected for multiplicity via the Holm–Bonferroni procedure.

Our principal findings are as follows. GM-P resolves 234 of 500 issues (46.8%, 95% CI [42.5%, 51.2%]), significantly outperforming every baseline in all nine pairwise comparisons after correction ($p < 0.05$). The oracle combination of all methods reaches 53.8%, indicating that the primary ceiling is seed selection rather than retrieval depth. GM-P's cost of ∼256K tokens per resolved issue is approximately 7.5× lower than RAG-Progressive (1.93M tokens), making it the most cost-effective method among the high-performing tier.[2] Methods such as BM25 are cheaper in absolute terms but resolve substantially fewer issues. We emphasize that our baselines are *controlled re-implementations* designed to isolate the effect of context-construction strategy; we do not claim to reproduce or surpass the full pipelines of Agentless, SWE-Agent, or Aider. The remainder of the paper is organized as follows: Section 2 surveys related work; Section 3 details the GraphManager framework and all compared strategies; Section 4 describes the experimental setup; Section 5 presents results and statistical analysis; Section 7 discusses threats to validity; and Section 8 concludes.

2 Background and Related Work

2.1 LLM-Based Automated Program Repair

Automated program repair (APR) has evolved from template-based and search-based techniques to LLM-driven approaches that treat patch generation as a conditional text-generation problem. Contemporary systems can be broadly categorized into three paradigms: (i) *generate-and-validate* methods that sample candidate patches and filter with test suites, (ii) *conversational or agentic* methods that iteratively explore the repository through tool use [27, 28], and (iii) *localize-then-patch* pipelines that first narrow the search space to a small set of files or functions before invoking the LLM for editing [26]. SWE-bench [12] operationalizes the repository-level repair task: given an issue description and a full repository snapshot, produce a patch that passes held-out tests. Its *Verified* subset, curated through human validation, reduces label noise and has become the primary leaderboard. A key observation across top-performing systems is that they share similar base LLMs; performance differences are driven predominantly by the *context-construction* strategy—i.e., which code the model sees and in what order. Recent multi-language extensions such as SWE-PolyBench [23] further underscore the generality of this bottleneck.

2.2 Retrieval-Augmented Generation for Code

Retrieval-augmented generation (RAG) [14] has become the default mechanism for injecting repository knowledge into an LLM's context window. The standard pipeline chunks source files at function or class granularity, embeds each chunk with a code-aware embedding model, indexes the vectors (e.g., with FAISS [13]), and at query time retrieves the top-k chunks most similar to the issue text. Variants differ in embedding model, chunking strategy, reranking, and whether retrieval is performed once (fixed-window) or iteratively (progressive). GraphRAG [5] augments the pipeline with entity-relationship graphs extracted from documents, while RepoHyper [22] and CodeGraphRAG [24] incorporate repository-specific structural signals into the retrieval index. Despite these advances, the cost structure of RAG—per-token embedding charges, index storage, and the need to re-index after every commit—is rarely quantified in SWE-bench evaluations. Moreover, semantic embeddings can suffer from *drift* on code: identifier names, boilerplate imports, and formatting artifacts may dominate the embedding space, reducing retrieval precision for the structural relationships that matter most for localization.

2.3 Structural and Graph-Based Code Representations

Programming languages expose rich structural information that predates the LLM era. Program dependence graphs, call graphs, and import graphs have long been used in software engineering for tasks such as impact analysis, slicing, and fault localization [18]. Tree-sitter [1] provides fast, incremental, language-agnostic parsing that makes it practical to extract function-level dependency edges from large polyglot repositories. RepoGraph [21] and GraphCoder [16] construct repository-level graphs and use them to guide code completion, while RepoScope [17] builds hierarchical structural summaries for navigation. Aider's `RepoMap` [6] computes a PageRank-weighted summary of the repository's tag graph to fit a concise structural overview into the LLM's context window. However, none of these systems have been eval-

[2] Both the dependency graph and the RAG vector index were prebuilt in our harness; the cost figures reflect method-specific LLM token consumption. We discuss this dual-build caveat in Section 7.

uated head-to-head against RAG baselines under controlled conditions on a standardized repair benchmark. Our work fills this gap: we construct a dependency graph using Tree-sitter, expose it to the LLM through agentic tool calls, and compare the resulting localization quality and cost against RAG and other strategies on SWE-bench Verified. We note that our *RepoMap-Like* baseline is a controlled re-implementation that applies PageRank on the import graph rather than a faithful reproduction of Aider's full system; similarly, our *Agentless-Like* baseline re-implements the hierarchical localization stages of Xia et al. [26] but collapses Stage 3 output to file-level granularity. These design choices are detailed in Section 3.

2.4 Cost-Aware Evaluation of LLM Systems

A growing body of work advocates for cost- and efficiency-aware evaluation of LLM-based software engineering tools. RepoBench [15] and RepoExec [9] measure execution-based metrics beyond textual match, and recent reliability studies of GraphRAG pipelines [3] highlight the sensitivity of retrieval quality to graph construction choices. Yet cost reporting remains inconsistent: some studies count API dollars, others count tokens, and many report neither. In this work we adopt *tokens consumed per resolved issue* as a reproducible, currency-agnostic cost metric and pair it with statistical significance testing to distinguish genuine quality differences from noise. To our knowledge, this is the first SWE-bench study to report Holm–Bonferroni-corrected McNemar tests across all pairwise method comparisons alongside per-issue cost breakdowns.

3 Method

We present a retrieval framework that operates over a structural representation of a software repository—a *graph world model*—and a patching pipeline that uses retrieved context to generate unified diffs. The framework supports eleven retrieval strategies spanning three tiers: zero-LLM-runtime baselines, single-turn agentic ablations, and multi-turn agentic methods. This section describes the graph world model (§3.1), the retrieval methods in each tier (§3.2), the patching pipeline (§3.3), and the cost model used for comparison (§3.4).

3.1 Graph World Model

Given a repository snapshot at a specific commit, we construct a directed, typed knowledge graph $G = (V, E)$ using static analysis of the Python source tree via the `tree-sitter` parser. Nodes V are categorised into three types: *file* nodes (one per source file), *class* nodes, and *function/method* nodes. Each node stores a compact metadata record comprising its qualified name, file path, docstring (if present), and a brief signature. Node embeddings are computed once at build time over this metadata using the Gemini text-embedding-004 model; the resulting vectors populate a FAISS IVF index for semantic search.

Edges E encode four structural relation types: CONTAINS (file → class, file → function, class → method), IMPORTS (file → file, resolved statically), CALLS (function → function, with confidence tier: *definite*, *probable*, or *possible* based on static resolvability), and INHERITS (class → parent class). The graph index is built once per repository snapshot; across a retrieval evaluation run on a single snapshot, the setup cost is amortised over all issues evaluated against that snapshot.

3.2 Retrieval Methods

We evaluate eleven retrieval strategies, grouped into three tiers by their use of LLM calls at query time.

3.2.1 Zero-LLM-Runtime Tier

These methods issue no LLM calls during query execution. Their total token cost consists only of the one-time graph setup embedding.

BM25 (`bm25`). BM25Plus [18] over tokenised source files provides a non-neural baseline with no index. Files are scored by term-overlap between the issue text and the concatenation of file path, function names, docstrings, and source tokens. No graph structure or embeddings are used.

Graph-PageRank (`repomap_like`). Loosely inspired by Aider's repository map concept [6], this method computes a personalised PageRank over the file-level import graph, treating files that have high BM25 similarity to the issue as seed nodes. The final file score is the product of the BM25 score and the PageRank mass, biasing retrieval toward structurally central files that are lexically relevant to the issue. **Note:** unlike Aider's original mechanism, which uses a compressed AST-based text representation of the repository, our adaptation replaces this with personalised PageRank over the static import graph. This is a controlled architectural comparison, not a faithful reproduction of the Aider system.

GM-Deterministic (`gm_deterministic`). A multi-component scoring function over all nodes in the graph world model, requiring no LLM calls at query time. For each node, a score is computed as a weighted sum of four components: (1) *semantic score*—cosine similarity between the query embedding and the node embedding; (2) *structural score*—a BM25 match of the issue text against the node's graph neighbourhood (immediate import/call neighbours); (3) *confidence score*—inversely proportional

to the minimum call-edge confidence tier on paths from the query seed to the node; and (4) *hub penalty*—a multiplicative discount applied to nodes with very high in-degree (hub nodes that appear spuriously relevant due to centrality). The top-K scored files are returned.

Raw-RAG-Function (`raw_rag_function`). Dense retrieval over function-body embeddings. Each function (or method) in the repository is embedded separately using its full source text. At query time, the top-K most similar function embeddings are retrieved and their containing files are returned. This establishes a fine-grained dense-retrieval baseline that does not use graph structure.

Raw-RAG-Fixed (`raw_rag_fixed`). Dense retrieval over fixed-size sliding-window chunks. The source code of each file is chunked into overlapping windows of a fixed token budget, each chunk embedded independently. Top-K chunks by cosine similarity are retrieved and their source files returned. This corresponds to the standard code-RAG approach used in production retrieval pipelines.

3.2.2 Single-Turn Agentic Tier (Ablation Variants)

These methods make a single LLM call at query time to either score or query the graph. They appear in the retrieval evaluation to isolate the contribution of multi-turn reasoning, but are excluded from patching runs (which require high-quality file lists).

GM-Baseline (`gm_baseline`). A single LLM turn is permitted; the model may invoke the same graph tools as GM-Progressive (`search_nodes`, `get_neighbors`, `get_file_summary`) but receives no follow-up turns. This ablation isolates the benefit of iterative multi-turn graph navigation over single-pass tool-assisted scoring.

RAG-Baseline (`rag_baseline`). A single LLM turn is permitted; the model may invoke one round of the same symmetric tools as RAG-Progressive (`semantic_search`, `get_file_contents`, `confirm_file`) but receives no follow-up turns. This ablation isolates the benefit of multi-turn query refinement in RAG-Progressive.

3.2.3 Multi-Turn Agentic Tier

These methods engage the LLM in an iterative dialogue with retrieval tools, allowing progressive refinement.

GM-Progressive (`gm_progressive`). The primary graph-guided method. The LLM is provided with a tool set for graph traversal: `search_nodes` (semantic search over the FAISS index), `get_neighbors` (expand a node's immediate graph neighbourhood), and `get_file_summary` (list all definitions in a file and implicitly mark it as confirmed). In each turn, the model may call any tool or emit a final answer. The dialogue continues until the model emits a final file list or a stagnation limit is reached (consecutive turns with no new file confirmations). The multi-turn structure allows the model to progressively build confidence in which files are relevant, using graph structure to resolve structural dependencies that embedding similarity alone cannot capture.

RAG-Progressive (`rag_progressive`). Multi-turn RAG using symmetric tools: `semantic_search` (dense vector search), `get_file_contents` (read a source file), and `confirm_file`. The model may issue multiple queries with refined terms and inspect file contents before committing to a final list. This provides a strong agentic baseline that benefits from multi-turn refinement without graph structure.

Agentless-Like (`agentless_like_localization`). A three-stage hierarchical localization procedure adapted from Agentless [26]. Stage 1: the LLM receives the directory tree and ranks candidate files by relevance to the issue. Stage 2: the LLM receives graph-derived symbol-level structure (functions, classes, and their signatures) for the Stage 1 candidates and selects relevant symbols. Stage 3: the LLM receives the selected symbol bodies and identifies the precise relevant spans, which are then mapped back to file-level granularity to conform to the unified file-list interface used across all methods. **Important:** unlike the original Agentless, our implementation uses the GraphManager dependency graph to supply symbol-level structure in Stage 2, making this a partially graph-augmented method. Additionally, our implementation generates a single repair candidate, whereas the original Agentless uses 40 samples with execution-based filtering. Results for this method should be interpreted with these differences in mind (see Section 7).

Agentic Cold-Start (`agentic_cold_start`). Direct filesystem tool use with no retrieval index. The LLM is provided with tools to navigate and read the raw repository: `list_directory`, `read_file`, and `search_code` (regex over file contents). No graph, no embedding index. This method serves as a baseline for the intrinsic file-navigation capability of the model and as an upper bound on what is achievable without retrieval infrastructure.

3.3 Patching Method

Patch generation is a single-turn procedure applied uniformly across all retrieval methods that produce a confirmed file list. The issue description and the full contents of the confirmed files are concatenated into a single context window, and a Gemini 2.0 Flash model generates a

unified diff in standard `git diff` format. No multi-turn repair is attempted. This design isolates the effect of retrieval quality on patch success: differences in resolved rate across retrieval methods reflect retrieval quality, not patching strategy variation.

Patch evaluation uses the SWE-bench Verified harness [12]: the generated patch is applied to the repository at the issue's `base_commit` and the repository's test suite is executed in an isolated Docker container. A patch is counted as resolved if and only if all tests that were failing before the patch now pass, with no regression in previously passing tests.

3.4 Cost Model

Token cost is decomposed into three components for every method:

Setup embedding tokens
Tokens consumed building the retrieval index (graph metadata embeddings, or code chunk embeddings for RAG methods). Amortised over all issues evaluated against a single snapshot.

Runtime LLM tokens
Prompt + completion tokens consumed by LLM calls during query execution. Zero for the Zero-LLM-Runtime tier.

Query embedding tokens
Tokens consumed embedding the issue query for dense search. Zero for BM25 and Graph-PageRank.

We report two cost-per-resolved-issue (CPR) views: *method-accounted CPR* adds each method's full setup cost to its runtime cost before dividing by resolved count, providing a like-for-like comparison; *as-run CPR* reports only the tokens actually consumed in the patching run (i.e., excluding setup costs shared with retrieval evaluation), reflecting practical amortised cost when the same index is reused.

4 Experimental Setup

4.1 Research Questions

We organize our evaluation around three research questions:

RQ1 (Effectiveness) Does GM-Progressive (GM-P) resolve more SWE-bench Verified instances than alternative context-construction methods under an identical downstream patching pipeline?

RQ2 (Cost) What is the per-resolved-issue token cost of GM-P relative to RAG-based and other retrieval methods?

RQ3 (Ceiling) How much headroom remains between GM-P and an oracle that selects the best-performing method per instance?

4.2 Benchmark and Instance Selection

We evaluate on **SWE-bench Verified** [11], a curated subset of 500 instances drawn from 12 open-source Python repositories spanning web frameworks (Django), scientific computing (scikit-learn, sympy, matplotlib), developer tooling (pytest, pylint, sphinx), and data processing (astropy, xarray, pydata/sparse, requests, flask). Each instance pairs a GitHub issue description with a ground-truth patch and a repository-specific test suite that must pass for the instance to be marked as resolved. The 500 instances exhibit substantial diversity in patch complexity: single-line fixes, multi-file refactorings, and additions of new functionality. Every method is executed on all 500 instances, yielding paired binary outcomes (resolved/not resolved) suitable for within-subject statistical testing.

4.3 Methods Compared

We compare 11 context-construction strategies organized into five families:

1. **Graph-based:** *GM-Progressive* (GM-P, proposed), which performs LLM-guided progressive expansion over the graph model; *GM-Deterministic* (GM-D), which uses fixed-depth breadth-first expansion without LLM ranking; and *RepoMap-Like* (RPM), a reimplementation of the tree-sitter-based repository skeleton approach inspired by Aider's RepoMap [7].

2. **RAG-based:** *RAG-Progressive* (RAG-P), the embedding-based analogue of GM-P that iteratively retrieves and expands context via vector similarity; *RAG-Metadata* (RAG-M), which enriches chunk embeddings with structural metadata; *Raw-RAG-Fixed* (RRX), a fixed-budget single-shot RAG retrieval; and *Raw-RAG-Function* (RRF), which retrieves at function-level granularity.

3. **Search-based:** *BM25*, a sparse lexical retrieval baseline using Okapi BM25 over function-level chunks.

4. **Agentic:** *Agentic-Cold-Start* (ACS), which employs an LLM agent with file-browsing tools to locate relevant context from scratch.

5. **Hybrid:** *Agentless-Like Localization* (AGL), which adapts the Agentless [26] two-stage localization pipeline. **Disclosure:** AGL uses the GM graph in its Stage 2 refinement step; this constitutes a confound, and its results should be interpreted with this dependency in mind.

Table 1 in the appendix provides a structured comparison of all methods along dimensions including context source, LLM usage during construction, and index reusability.

4.4 Downstream Patching Pipeline

To isolate the effect of context construction, all 11 methods feed into an *identical* downstream patching pipeline adapted from Agentless [26]. The pipeline proceeds in two stages: (1) *localization*, where the constructed context is presented to the LLM alongside the issue description to identify candidate edit locations, and (2) *patch generation*, where the LLM produces a unified diff targeting the identified locations. We use GPT-4o (`gpt-4o-2024-05-13`) with temperature 0.0 and a maximum output length of 4,096 tokens for both stages. Generated patches are validated using the SWE-bench Docker harness, which executes the repository's full test suite against the patched code. An instance is marked as resolved only if all relevant tests pass. The sole independent variable across experimental conditions is the context-construction strategy; the LLM, prompt templates, patch generation logic, and validation harness are held constant.

4.5 Cost Accounting Methodology

We report cost in terms of *tokens consumed*, encompassing both input and output tokens across all LLM and embedding API calls. Our primary metric is **method-accounted cost**: only the API and compute costs attributable to the method's design-intended execution path. For graph-based methods, this includes graph construction (deterministic, LLM-free) and any LLM calls during progressive expansion. For RAG-based methods, this includes embedding generation for all chunks and vector retrieval queries.

Dual-build disclosure. Due to an architectural coupling in the experimental harness (`run_patch.py`), both the graph index and the RAG embedding index were constructed for every experimental run, regardless of which method was under evaluation. We report *method-accounted* cost (reflecting design intent) as the primary metric throughout the paper. The full *as-run-consumed* cost, which includes the redundant index construction, is reported in Appendix A for transparency. The dual-build overhead does not affect resolve rates or the relative ordering of methods on the cost axis, as it is constant across all conditions.

Amortization. Graph and embedding index construction costs are amortized across all issues sharing the same repository snapshot. As issue volume per snapshot increases, the marginal cost of graph-based methods approaches only the per-issue expansion cost, which is substantially lower than the per-issue retrieval cost of RAG methods.

4.6 Statistical Testing Protocol

We use **McNemar's exact test** [20] on paired binary outcomes (resolved/not resolved) for each of the nine pairwise comparisons between GM-P and every other method. McNemar's test is appropriate here because the 500 instances are shared across conditions, and the test statistic depends only on the discordant pairs—instances where exactly one method succeeds. We apply **Holm–Bonferroni correction** [10] across the nine comparisons to control the family-wise error rate at $\alpha = 0.05$. Effect sizes are reported as **Cohen's** h [4], computed from the arcsine-transformed proportions, with conventional thresholds of 0.2 (small), 0.5 (medium), and 0.8 (large). Confidence intervals for resolve rates are **Wilson score intervals** [25] at the 95% level.

Limitations of the testing protocol. Each method was executed in a *single run* per instance. McNemar's test on $N = 500$ paired binary observations provides adequate statistical power for the effect sizes observed (the smallest significant $h = 0.10$ corresponds to 67 vs. 43 discordant pairs). However, a single run does not capture variance arising from LLM sampling stochasticity (e.g., temperature-induced variation, API-side nondeterminism). We flag this as a limitation: observed differences at the lower end of the effect-size range (e.g., GM-P vs. RRX, $h = 0.10$) should be interpreted with caution, as they might not replicate under repeated sampling. Additionally, the 500 instances are drawn from 12 repositories, introducing within-repository clustering that McNemar's test does not model. We report per-repository breakdowns in Appendix A to assess the consistency of effects across repositories.

5 Results

5.1 Overall Resolve Rates (RQ1)

Table 1 presents the resolve rates for all 11 methods on SWE-bench Verified. GM-P resolves **234 of 500 instances (46.8%)**, the highest rate among all non-oracle methods. The full rank ordering is: Oracle (53.8%) > GM-P (46.8%) > RRX (42.0%) > RAG-P (41.6%) > RRF (40.6%) > BM25 (39.6%) > ACS (36.8%) > RAG-M (34.8%) > GM-D (32.6%) > AGL (22.4%) > RPM (6.6%).

The absolute difference between GM-P and the next-best method (RRX) is 4.8 percentage points (24 additional resolved instances). The gap widens substantially for methods further down the ranking: GM-P resolves 14.2 points more than GM-D and 24.4 points more than AGL.

Table 1: Resolve rates on SWE-bench Verified (500 instances, 12 repositories). Wilson 95% CIs in brackets. McNemar tests compare each method against GM-P; all survive Holm–Bonferroni correction at $\alpha = 0.05$.

Method	Res.	Rate	[95% CI]	p_{Holm}	h
Oracle	269	53.8%	[49.4%, 58.1%]	—	—
GM-P	**234**	**46.8%**	[42.5%, 51.2%]	—	—
RRX	210	42.0%	[37.8%, 46.4%]	0.028	0.10
RAG-P	208	41.6%	[37.4%, 46.0%]	0.036	0.10
RRF	203	40.6%	[36.4%, 45.0%]	0.013	0.13
BM25	198	39.6%	[35.4%, 44.0%]	0.007	0.15
ACS	184	36.8%	[32.7%, 41.1%]	$<.001$	0.20
RAG-M	174	34.8%	[30.8%, 39.1%]	—	0.24
GM-D	163	32.6%	[28.6%, 36.8%]	—	0.29
AGL	112	22.4%	[19.0%, 26.3%]	—	0.52
RPM	33	6.6%	[4.7%, 9.1%]	—	0.99

Note on RepoMap-Like. RPM achieved only 6.6% (33/500), far below all other methods. We believe this reflects limitations in our reimplementation rather than a definitive assessment of the repository-map concept. The tree-sitter-based skeleton may require integration-specific tuning that our reproduction did not capture. We include RPM for completeness but do not draw strong conclusions about the RepoMap approach from this result.

5.2 Statistical Significance (RQ1 continued)

All nine pairwise McNemar tests comparing GM-P against each alternative method are statistically significant after Holm–Bonferroni correction ($p_{\text{Holm}} < 0.05$ for all). GM-P achieved a higher resolve rate than every compared method, with paired differences significant under McNemar's test with Holm–Bonferroni correction.

The effect sizes span a wide range. The largest effect is observed against RPM ($h = 0.99$, large), followed by AGL ($h = 0.52$, medium). Against the stronger baselines, effect sizes are smaller: $h = 0.15$ for BM25, $h = 0.10$ for both RRX and RAG-P. While statistically significant, these smaller effects correspond to modest absolute differences (e.g., 24 instances between GM-P and RRX) and should be interpreted with appropriate caution given the single-run design.

The discordant-pair structure is informative. Against AGL, GM-P uniquely resolves 141 instances while AGL uniquely resolves only 19 (b/c ratio of 7.4:1). Even against the competitive RRX baseline, GM-P uniquely resolves 67 instances versus 43 for RRX ($b/c = 1.6$:1). This asymmetry indicates that GM-P's advantage is not merely a uniform uplift but reflects the ability to solve qualitatively different instances—a finding that motivates the oracle analysis in §5.4.

Table 2: Cost per resolved issue (CPR) in tokens (method-accounted). Median across repositories with ≥ 1 resolved instance.

Method	Median CPR	Ratio vs. GM-P
Oracle	23,774	0.1×
GM-P	**256,821**	**1.0×**
RRX	1,464,260	5.7×
RAG-P	1,932,577	7.5×
RRF	2,194,208	8.5×
BM25	179,990	0.7×
ACS	122,761	0.5×
RAG-M	652,995	2.5×
GM-D	488,102	1.9×
AGL	2,067,066	8.0×
RPM	598,542	2.3×

5.3 Cost Comparison (RQ2)

Table 2 reports the median cost per resolved issue (CPR) in tokens, computed across repositories with at least one resolved instance.

GM-P's median CPR of 256,821 tokens is approximately 7.5× lower than RAG-P (1,932,577 tokens) and 8.5× lower than RRF (2,194,208 tokens). The cost advantage stems from two structural properties of the graph-based approach: (1) graph construction is deterministic and LLM-free, requiring only static analysis, whereas RAG indexing requires embedding API calls proportional to the number of code chunks; and (2) progressive expansion in GM-P invokes the LLM only to rank a small set of structurally adjacent candidate nodes, rather than to search the entire repository embedding space.

Two methods—BM25 (0.7×) and ACS (0.5×)—achieve lower CPR than GM-P. However, both resolve substantially fewer instances (198 and 184, respectively, versus 234 for GM-P). BM25's low cost reflects the absence of LLM calls during retrieval, while ACS benefits from early termination when the agent identifies a plausible location quickly. Neither method matches GM-P's resolve rate, and the cost savings come at the expense of 36–50 fewer resolved instances.

Considering both effectiveness and cost jointly, **GM-P occupies the Pareto frontier**: no other method achieves both a higher resolve rate and a lower cost per resolved issue. Methods that are cheaper (BM25, ACS) resolve fewer issues; methods that resolve a comparable number of issues (RRX, RAG-P) cost 5.7–7.5× more per resolution.

5.4 Oracle Analysis (RQ3)

The per-instance oracle, which selects the best-performing method for each instance, resolves **269 of 500 instances (53.8%)**. The gap between GM-P (46.8%) and the oracle (53.8%) is 7.0 percentage points, corre-

sponding to 35 additional instances that at least one alternative method resolves but GM-P does not.

This gap confirms that no single context-construction strategy dominates across all instances. Qualitative inspection of the 35 oracle-only instances reveals two recurring patterns: (1) instances requiring broad semantic context spanning multiple loosely coupled modules, where RAG-based methods' ability to retrieve by textual similarity proves advantageous; and (2) instances involving boilerplate or configuration-heavy code where BM25's lexical matching suffices and the graph structure provides little additional signal. These complementary strengths suggest that an ensemble or adaptive method-selection strategy could close a substantial portion of the oracle gap, a direction we discuss further in §6.

5.5 Per-Repository Consistency

GM-P's advantage is broadly consistent across the 12 repositories, though the magnitude varies. GM-P achieves the highest or tied-highest resolve rate in 9 of 12 repositories. In the remaining three, the margin between GM-P and the top method is within 1–3 instances. We observe that GM-P's advantage is most pronounced in repositories with deep dependency structures (e.g., Django, sympy), where multi-hop structural traversal surfaces context that flat retrieval methods miss. Full per-repository results are provided in Appendix A.

6 Discussion

6.1 Why Does Structural Context Work Well?

The strong performance of GM-P relative to embedding-based alternatives can be attributed to a fundamental property of source code: *dependencies are explicit and deterministic*. Import statements, function calls, class inheritance hierarchies, and module boundaries are syntactically recoverable through static analysis without requiring learned representations. For bug-fixing tasks, the relevant context is overwhelmingly *structurally adjacent* to the fault locale—callers and callees of the buggy function, sibling methods in the same class, overridden methods in parent classes, and co-imported utility functions.

Embedding-based retrieval, by contrast, operates on distributional similarity in a learned vector space. While this can surface semantically related code, it may also retrieve code that is topically similar but structurally irrelevant—for example, a different module's error-handling routine that uses similar vocabulary but shares no dependency relationship with the fault. Such false positives consume the LLM's finite context window without contributing to patch generation. The graph model's progressive expansion avoids this failure mode by construction: every node added to the context is reachable via an explicit dependency edge from the seed locale.

The comparison between GM-P and GM-D (46.8% vs. 32.6%) further isolates the contribution of *LLM-guided* expansion. The graph structure alone (GM-D) provides a useful skeleton, but the LLM's ability to rank candidate nodes by relevance to the specific issue—effectively pruning irrelevant branches of the dependency graph—yields a 14.2-point improvement. This suggests that the combination of structural constraints (limiting *where* to look) and semantic judgment (deciding *what* to include) is more effective than either alone.

6.2 Cost Efficiency in Depth

The cost advantage of GM-P has both a *fixed* and a *variable* component. The fixed component—graph construction via static analysis—requires no LLM or embedding API calls and scales linearly with repository size. The variable component—LLM-guided progressive expansion—involves a small number of ranking calls per issue, typically 3–5 iterations with candidate sets of 10–20 nodes each. By contrast, RAG-based methods incur a fixed cost proportional to the total number of code chunks (for embedding generation) and a variable cost per query that scales with the retrieval depth and re-ranking strategy.

In a deployment scenario such as CI/CD integration, where many issues arise against the same repository snapshot, the graph construction cost is amortized across all issues. As issue volume grows, GM-P's marginal cost per issue approaches only the expansion LLM calls, which are substantially cheaper than the per-issue embedding retrieval costs of RAG methods. This amortization dynamic makes the graph-based approach increasingly favorable at scale.

We reiterate the dual-build disclosure from §4: the cost figures in Table 2 reflect method-accounted attribution. In the actual experimental runs, both graph and RAG indices were constructed for every condition due to an architectural coupling in the harness. The as-run-consumed costs, reported in Appendix A, are uniformly higher but do not change the relative ordering of methods.

6.3 Complementarity and the Oracle Gap

The 7.0-percentage-point gap between GM-P and the oracle (53.8%) represents 35 instances where at least one alternative method succeeds but GM-P does not. This gap is practically significant: closing it would represent a 15% relative improvement over GM-P's current resolve rate.

Analysis of the discordant instances reveals that the methods' failure modes are largely complementary. Instances where RAG-based methods succeed and GM-P

fails tend to involve: (a) cross-cutting concerns that span structurally distant modules connected by semantic similarity rather than explicit dependencies, and (b) issues whose descriptions use domain-specific terminology that aligns well with code comments and docstrings captured by embeddings but not by the dependency graph. Conversely, instances where GM-P succeeds and RAG fails typically involve: (a) deeply nested dependency chains where the relevant context is 3–5 hops from the fault locale, and (b) issues requiring understanding of class hierarchies or interface contracts that are explicit in the graph but diluted in embedding space.

These complementary strengths suggest a promising direction for future work: an *adaptive* or *ensemble* strategy that selects or combines context-construction methods based on instance characteristics. A lightweight classifier trained on issue metadata (e.g., number of files mentioned, presence of stack traces, repository structural properties) could route instances to the most appropriate method, potentially approaching the oracle's 53.8% resolve rate.

6.4 Interpreting Agentless-Like Localization

AGL's resolve rate of 22.4% is notably lower than GM-P's 46.8%, but this comparison requires careful interpretation. As disclosed in §4, AGL's Stage 2 refinement uses the GM graph to narrow candidate locations. This means AGL is not a fully independent baseline: its performance is partially dependent on the same graph infrastructure that underlies GM-P. The low resolve rate may reflect suboptimal integration of graph information within the Agentless localization framework rather than a fundamental limitation of the Agentless approach. We include AGL for completeness but caution against interpreting the GM-P vs. AGL comparison as evidence that graph-based context is universally superior to Agentless-style localization.

6.5 Threats to Validity

Internal validity. The single-run design does not capture LLM sampling variance. While McNemar's test on 500 paired observations provides adequate power for the observed effect sizes, the smallest significant effects ($h = 0.10$ for RRX and RAG-P) are close to the boundary of practical significance and might not replicate under repeated sampling with nonzero temperature. Future work should conduct multiple runs per method to estimate confidence intervals that account for sampling stochasticity.

Construct validity. Within-repository clustering may inflate the effective sample size: instances from the same repository share codebase characteristics, dependency structures, and coding conventions. McNemar's test treats all 500 instances as exchangeable, which may understate uncertainty. The per-repository analysis in Appendix A provides a partial check, but a mixed-effects model accounting for repository-level random effects would be more rigorous.

External validity. SWE-bench Verified covers 12 Python repositories. The generalizability of our findings to other languages, repository sizes, or issue types (e.g., feature requests, security vulnerabilities) remains untested. The graph model's reliance on static analysis may be less effective for dynamically typed languages with heavy metaprogramming or for repositories with unconventional module structures.

Reproducibility. The dual-build artifact in the experimental harness, while not affecting resolve rates, complicates exact cost reproduction. We release the full experimental configuration and logs to enable independent verification.

7 Threats to Validity

7.1 Internal Validity

Single run per method. LLM outputs are stochastic; our single-run design does not capture sampling variance. McNemar's test is valid for the observed paired outcomes across 500 instances, but the resolve rates and pairwise significance results could shift under repeated sampling. We consider this the most significant internal threat. Future work should employ multi-run designs with variance estimation to strengthen statistical claims.

Dual-build confound. The experimental harness constructed both the dependency graph and the RAG vector index for every run, regardless of which retrieval strategy was under evaluation. While cost is reported on a method-accounted basis (Section 5), subtle runtime interactions—such as filesystem caching, memory pressure from concurrent index structures, or I/O contention—could in principle affect downstream LLM latency or behavior. We found no evidence of such effects in our logs but cannot categorically rule them out.

Agentless-Like confound. Agentless-Like Localization uses the GM dependency graph during its Stage 2 localization step, making it partially a graph-based method. Its lower resolve rate relative to GM-P, despite access to the same graph, suggests that *how* the graph is traversed matters as much as *whether* a graph is available. However, this shared dependency means comparisons involving Agentless-Like do not isolate the original Agentless localization strategy [26] and should be interpreted with this confound in mind.

7.2 External Validity

Benchmark and language scope. SWE-bench Verified comprises 500 instances drawn from 12 Python repositories. Python's dynamic type system and explicit `import` mechanism may favor or disfavor graph-based approaches relative to statically-typed languages (e.g., Java, TypeScript) where richer type-level dependency information is available at parse time. Results may not generalize to other languages, repository sizes, issue types (e.g., feature requests versus bug fixes), or benchmarks.

Within-repo clustering. The 500 instances are drawn from only 12 repositories, with highly uneven representation: Django alone contributes 231 instances (46.2%), while several repositories contribute fewer than 25 (see Table 3 in the Appendix). Instances from the same repository share codebase structure, coding conventions, and dependency patterns, violating the IID assumption. McNemar's test conditions on paired outcomes rather than assuming independence across instances, partially mitigating this concern, but repository-level confounds (e.g., a method that happens to suit Django's architecture) could inflate aggregate resolve rates.

LLM dependence. All methods use the same downstream LLM for patch generation. Results may differ substantially with other models, particularly those with different context-window sizes, code-understanding capabilities, or instruction-following fidelity.

7.3 Construct Validity

Resolve rate as sole effectiveness metric. We evaluate only end-to-end resolve rate (i.e., whether the generated patch passes the held-out test suite). This metric conflates retrieval quality with patch-generation quality: a method may retrieve ideal context yet fail at patch synthesis, or succeed with mediocre context due to LLM robustness. We did not execute the retrieval-evaluation track (T0), so retrieval-level metrics such as file-level or function-level F1 are unavailable. This limits our ability to diagnose *why* methods succeed or fail.

Cost metric limitations. Method-accounted cost is an idealized measure that attributes only the API and index-construction costs directly incurred by each strategy. Actual deployment cost depends on infrastructure choices, caching policies, request batching, and workload patterns not captured in our experimental setup.

7.4 Reproducibility

Proxy implementations. Several baselines are named with the "-Like" suffix to signal that they are proxy reimplementations rather than the original systems. In particular, the RepoMap-Like baseline achieved only 6.6% resolve rate—far below the performance reported for the Aider system from which it draws inspiration [7]. This result most likely reflects an implementation deficiency (e.g., incorrect tag-map construction or context-window formatting) rather than a fundamental limitation of repository-map approaches. We include it for transparency but caution against interpreting this data point as evidence that structural maps are ineffective in general. Faithful reimplementation of context-construction strategies is non-trivial; community-standard reference implementations would benefit the field.

LLM non-determinism. All code and configuration will be released upon publication. However, exact LLM outputs are inherently non-reproducible due to API-side model updates, stochastic decoding, and potential changes to serving infrastructure over time.

The most significant threat is the single-run design: while McNemar's test is valid for the observed paired outcomes, it does not account for variance introduced by LLM sampling across repeated runs. We encourage future work to adopt multi-sample protocols to quantify this variance.

8 Conclusion

We presented GM-Progressive (GM-P), a graph-based context-construction strategy for LLM-driven repository-level program repair. GM-P extracts a dependency graph from source code using lightweight static analysis, then traverses it with a progressive expansion policy to assemble context that is both structurally relevant and budget-aware.

On 500 SWE-bench Verified instances spanning 12 Python repositories, GM-P achieved a 46.8% resolve rate—significantly higher than all 10 compared methods after Holm–Bonferroni correction at $\alpha = 0.05$ (Section 5). The smallest observed effect (versus RAG-Progressive) corresponded to Cohen's $h = 0.105$, and the largest (versus RepoMap-Like) to $h = 0.987$. Critically, GM-P achieved this performance at approximately 8× lower per-resolved-issue cost than the best-performing RAG-based alternative (RAG-Progressive), because graph construction requires only a single-pass parse rather than embedding every code unit through an LLM API.

An oracle analysis combining the predictions of all methods revealed a 53.8% ceiling—7 percentage points above GM-P—indicating meaningful complementarity. Error analysis showed that RAG-based methods occasionally resolve instances that GM-P misses, particularly those involving semantically related but structurally distant code. This suggests that hybrid strategies combining graph-first retrieval with embedding-based fallback could be a productive direction for future work.

The central finding is that *explicit structural dependencies*, cheaply extracted from source code via tree-sitter [2], provide context that is at least as effective as learned embeddings for guiding LLM-based repair—and dramatically cheaper to obtain. This challenges the prevailing

assumption that retrieval-augmented generation is the necessary substrate for repository-scale LLM tasks, and motivates treating cost-per-resolved-issue as a first-class evaluation metric alongside resolve rate.

Several directions remain open. First, **hybrid strategies** that combine graph-based and embedding-based context—for example, graph-first retrieval with RAG fallback for low-confidence instances—could close the oracle gap. Second, **incremental graph maintenance**, updating the dependency graph on each commit rather than rebuilding per snapshot, could further reduce amortized cost in high-issue-volume settings such as large monorepos or nightly CI pipelines; we note this as a practical advantage but have not empirically validated it. Third, **cross-language generalization** to statically-typed languages (Java, TypeScript) where richer type information may improve graph quality warrants investigation. Finally, **multi-run evaluation** with multiple LLM samples per instance is needed to quantify sampling variance and place confidence intervals on the resolve-rate estimates reported here.

Structural dependency graphs offer a cost-effective and statistically reliable alternative to embedding-based retrieval for repository-level code repair. We release all code, graph-construction tooling, and experimental configurations to support replication and extension, and we encourage the community to adopt cost-per-resolved-issue as a standard metric in future evaluations of LLM-based software engineering tools.

References

[1] Max Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools. https://github.com/tree-sitter/tree-sitter, 2018.

[2] Max Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools. https://github.com/tree-sitter/tree-sitter, 2018.

[3] Manideep Reddy Chinthareddy. Reliable graph-RAG for codebases: AST-derived graphs vs LLM-extracted knowledge graphs. *arXiv preprint arXiv:2601.08773*, 2026.

[4] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.

[5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[6] Paul Gauthier. Aider: AI pair programming in your terminal. https://aider.chat, 2023.

[7] Paul Gauthier. Aider: Repository map. https://aider.chat/docs/repomap.html, 2024.

[8] Gemini Team, Google DeepMind. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2024.

[9] Nam Le Hai, Dung Nguyen Manh, and Nghi D. Q. Bui. RepoExec: Evaluate code generation with a repository-level executable benchmark. *arXiv preprint arXiv:2406.11927*, 2024. URL https://arxiv.org/abs/2406.11927.

[10] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6 (2):65–70, 1979.

[11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

[12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.06770.

[13] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wentau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[15] Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking repository-level code autocompletion systems. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2306.03091.

[16] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. GraphCoder: Enhancing repository-level code completion via coarse-to-fine retrieval based on code context graph. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.

[17] Yang Liu, Li Zhang, Fang Liu, Zhuohang Wang, Donglin Wei, Zhishuo Yang, Kechi Zhang, Jia Li, and Lin Shi. RepoScope: Leveraging call chain-aware multi-view context for repository-level code generation. *arXiv preprint arXiv:2507.14791*, 2025.

[18] Yuanhua Lv and ChengXiang Zhai. When documents are very long, BM25 fails! *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1103–1104, 2011.

[19] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[20] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[21] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. *arXiv preprint arXiv:2410.14684*, 2024.

[22] Huy Nhat Phan, Hoang Nhat Phan, Tien N. Nguyen, and Nghi D. Q. Bui. RepoHyper: Search-expand-refine on semantic graphs for repository-level code completion. In *Proceedings of the 2nd ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*, 2025. arXiv:2403.06095.

[23] Muhammad Shihab Rashid et al. SWE-PolyBench: A multi-language benchmark for repository-level evaluation of coding agents. *arXiv preprint arXiv:2504.08703*, 2025. URL https://arxiv.org/abs/2504.08703.

[24] vitali87. Code-graph-RAG. https://github.com/vitali87/code-graph-rag, 2025.

[25] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[26] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[27] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

[28] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.

A Per-Repository Breakdown

Table 3 reports the resolve counts and rates for GM-Progressive and selected comparison methods, broken down by repository. Repositories vary substantially in size, domain, and the number of contributed instances; Django alone accounts for 46.2% of the benchmark.

Several patterns are notable. GM-P performs strongest on pytest (63.2%), scikit-learn (59.4%), and django (56.3%)—repositories with well-structured module hierarchies and explicit import chains that the dependency graph captures effectively. Performance is weakest on sphinx (18.2%) and sympy (34.7%), where issues often involve deeply nested symbolic transformations or documentation-build logic that may not be well-represented by call-level and import-level edges. The oracle gap is largest for sympy (21.3 percentage points), suggesting that this repository would benefit most from hybrid or ensemble strategies.

B Full McNemar Test Results

Table 4 reports the complete pairwise McNemar test results for GM-Progressive versus each of the 10 comparison methods. All tests use the exact binomial variant of McNemar's test on the 2×2 contingency table of paired outcomes. Columns b and c denote the off-diagonal counts: b is the number of instances resolved by GM-P but not by the comparison method, and c is the converse. The raw p-value is from the two-sided exact McNemar test; p_{Holm} is the Holm–Bonferroni-adjusted p-value controlling the family-wise error rate across all 9 comparisons at $\alpha = 0.05$. Cohen's h is computed from the marginal proportions as a standardized effect size.

Several observations merit discussion. First, the comparison with RepoMap-Like ($h = 0.987$) is an extreme outlier driven by the likely implementation deficiency discussed in Section 7; this effect size should not be interpreted as representative of the gap between graph-based and map-based approaches in general. Second, the three closest comparisons—BM25, Raw-RAG-Function, RAG-Progressive, and Raw-RAG-Fixed—all have $h < 0.15$, indicating small but statistically significant effects. These small effect sizes underscore that the practical advantage of GM-P over strong RAG baselines lies primarily in *cost efficiency* rather than large resolve-rate differences. Third, the Agentless-Like comparison ($h = 0.521$, a medium-to-large effect) should be interpreted with the confound noted in Section 7: this method shares the GM

Table 3: Per-repository resolve counts (resolved / total) and resolve rates (%) for GM-Progressive and selected methods on SWE-bench Verified. Best resolve rate per repository is **bolded**. Oracle denotes the union of all 11 methods.

Repository	n	GM-P	Oracle	RAG-P	BM25	RRF-Hybrid	ACS	RRX
astropy	22	12 (54.5%)	11 (50.0%)	6 (27.3%)	8 (36.4%)	—	—	—
django	231	130 (56.3%)	143 (61.9%)	126 (54.5%)	—	—	—	131 (56.7%)
matplotlib	34	11 (32.4%)	10 (29.4%)	—	—	—	7 (20.6%)	—
pytest	19	12 (**63.2%**)	13 (68.4%)	11 (57.9%)	—	—	—	—
scikit-learn	32	19 (59.4%)	21 (**65.6%**)	18 (56.3%)	—	—	—	—
sympy	75	26 (34.7%)	42 (**56.0%**)	—	—	—	—	25 (33.3%)
sphinx	44	8 (18.2%)	11 (25.0%)	—	—	9 (**20.5%**)	—	—
xarray	22	12 (54.5%)	14 (**63.6%**)	11 (50.0%)	—	—	—	—
All	**500**	**234 (46.8%)**	**269 (53.8%)**	—	—	—	—	—

Dashes (—) indicate that the method was not among the top performers for that repository or that per-repo counts were not recorded for all methods. Oracle counts may appear lower than GM-P for individual repositories (e.g., astropy, matplotlib) because the oracle is computed instance-wise across all 11 methods; a repository-level count can be lower when the oracle resolves different instances than GM-P within that repository.

Table 4: Full McNemar test results: GM-Progressive vs. each comparison method ($n = 500$ paired instances). All comparisons are significant after Holm–Bonferroni correction at $\alpha = 0.05$.

Comparison Method	b	c	p (raw)	p_{Holm}	Cohen's h	Sig.
RepoMap-Like	212	11	$< 10^{-6}$	$< 10^{-6}$	0.987	Yes
Agentless-Like Loc.	141	19	$< 10^{-6}$	$< 10^{-6}$	0.521	Yes
GM-Deterministic	101	30	$< 10^{-6}$	$< 10^{-6}$	0.291	Yes
RAG-Metadata	94	34	$< 10^{-6}$	$< 10^{-6}$	0.245	Yes
Agentic Cold Start	87	37	4.0×10^{-6}	4.0×10^{-5}	0.203	Yes
BM25	81	45	7.6×10^{-4}	6.8×10^{-3}	0.146	Yes
Raw-RAG-Function	71	40	1.8×10^{-3}	1.3×10^{-2}	0.125	Yes
RAG-Progressive	69	43	5.9×10^{-3}	3.6×10^{-2}	0.105	Yes
Raw-RAG-Fixed	67	43	4.0×10^{-3}	2.8×10^{-2}	0.097	Yes

b: instances resolved by GM-P but not by the comparison method. c: instances resolved by the comparison method but not by GM-P. Rows are sorted by descending effect size. The Holm–Bonferroni procedure orders the 9 raw p-values and applies step-down adjustment; all remain below $\alpha = 0.05$.

dependency graph, so the comparison reflects differences in traversal strategy rather than graph-based versus non-graph-based retrieval.

The consistency of significance across all nine comparisons, surviving Holm–Bonferroni correction, provides confidence that GM-P's advantage is robust for the observed run, while the single-run limitation (Section 7) remains the primary caveat for generalization. "'